LABORATORIO DI INFORMATICA

Modulo M2

Fondamenti del linguaggio Python

Esercizi

Istituto Tecnico Tecnologico "G. Marconi" — Rovereto
Indirizzo Informatica e Telecomunicazioni, articolazione Informatica
Classe 4Bi — Anno scolastico 2026-2027 · Docente di laboratorio: Andrea Trentini

Modulo M2 — Esercizi sui fondamenti del linguaggio Python

Gli esercizi si svolgono sul Jupyter server d'istituto, in un unico notebook `esercizi_M2.ipynb` che al termine del modulo viene scaricato e portato nel repository personale, nella cartella `M2_python_fondamenti/`. Il notebook ha una cella Markdown di titolo e una cella di codice per ciascun esercizio, nell'ordine in cui gli esercizi sono elencati qui. Il lavoro si consegna con un `commit` per esercizio o per gruppo di esercizi svolti nella stessa lezione; il messaggio rispetta la convenzione fissata nel modulo M0, cioè una prima riga di massimo cinquanta caratteri al presente indicativo e senza punto finale, e richiama fra parentesi il numero dell'esercizio: per esempio `Aggiunge la somma delle cifre (M2 es. 7)`.

Prima della consegna il notebook va riavviato ed eseguito per intero, come si è visto nel modulo M1: una cella che funziona solo grazie a una variabile lasciata in memoria da un tentativo precedente non è un esercizio risolto. Ogni programma va provato almeno con i dati di esempio riportati nella consegna e con un caso limite scelto da chi lo scrive. L'output atteso indicato negli esercizi è quello che il programma deve produrre con i dati di esempio: gli spazi e i decimali contano.

Vincoli validi per tutti gli esercizi. Si usano soltanto i costrutti del modulo M2: tipi fondamentali, operatori, conversioni, `input` e `print`, f-string, `if` / `elif` / `else`, `while`, `for` con `range`, `break`, `continue`, la clausola `else` dei cicli, le stringhe con indicizzazione, slicing e metodi. Non sono ammesse liste, tuple, dizionari e insiemi come strutture di dati, che appartengono al modulo M3, né la definizione di funzioni con `def`, che appartiene al modulo M4; le funzioni predefinite come `len`, `int`, `float`, `str`, `round`, `abs` e `bin` si usano liberamente. I metodi che restituiscono una lista, in particolare `split`, non sono ammessi. Resta invece ammesso l'assegnamento multiplo nella forma vista nella dispensa, per esempio `x, y = y, x % y`, che è l'unica anticipazione del modulo M3 consentita.

Ogni cella di codice inizia con un commento di una riga che descrive il programma, e la cella Markdown che la precede riporta il numero e il titolo dell'esercizio. I nomi delle variabili seguono la convenzione PEP 8, cioè minuscole con la sottolineatura a separare le parole. L'indentazione è di quattro spazi.

Livello base

Esercizio 1 — Durata di una sessione di laboratorio

Scrivere nella cella dell'esercizio 1 il programma che legge in ingresso la durata di una sessione di laboratorio espressa in secondi e la stampa nel formato ore, minuti e secondi, usando soltanto la divisione intera e il resto. Il programma stampa anche la stessa durata espressa in

ore decimali con due cifre dopo la virgola, e segnala con un messaggio il caso in cui la durata superi le quattro ore.

Input di esempio: `9296`.

Output atteso:

```
02:34:56
in ore decimali: 2.58
```

Con l'ingresso `15000` va stampata anche la riga `attenzione: la sessione supera le quattro ore`.

Tempo stimato: 20 minuti

Esercizio 2 — Media dei voti e giudizio

Scrivere nella cella dell'esercizio 2 il programma che legge in ingresso il nome di uno studente, tre voti in virgola mobile e le ore di assenza, calcola la media dei tre voti e stampa il giudizio secondo questa regola: se le ore di assenza superano 50 il giudizio è `non valutabile`, indipendentemente dalla media; altrimenti il giudizio è `ottimo` da 8 in su, `sufficiente` da 6 in su, `insufficiente` sotto il 6. La media va stampata con due decimali dentro una f-string, insieme al nome dello studente.

Input di esempio: `Chiara`, `7.5`, `8`, `6.5`, `12`.

Output atteso:

```
Chiara: media 7.33, giudizio sufficiente
```

Tempo stimato: 25 minuti

Esercizio 3 — Tavola di conversione incolonnata

Scrivere nella cella dell'esercizio 3 il programma che legge in ingresso un valore iniziale, un valore finale e un passo, tutti interi, e stampa la tavola di conversione da chilometri a miglia e a metri per tutti i valori dell'intervallo, estremi compresi. Un chilometro corrisponde a 0,621371 miglia. La tabella ha un'intestazione e una riga di trattini, le colonne sono allineate a destra e i valori in miglia hanno tre decimali. Se il passo non è positivo il programma stampa un messaggio e non produce la tabella.

Input di esempio: `5`, `20`, `5`.

Output atteso:

```
        km       miglia        metri
-----------------------------------
         5        3.107         5000
        10        6.214        10000
        15        9.321        15000
        20       12.427        20000
```

Tempo stimato: 30 minuti

Esercizio 4 — Lettura di un codice di postazione

Scrivere nella cella dell'esercizio 4 il programma che legge in ingresso un codice di postazione di laboratorio nel formato `LAB2-PC17-2026`, dove le prime quattro posizioni indicano il laboratorio, le quattro successive al trattino indicano il calcolatore e le ultime quattro l'anno di installazione. Il programma ripulisce l'ingresso dagli spazi agli estremi, lo porta in maiuscolo ed estrae con lo slicing le tre parti, stampandole su tre righe distinte. Stampa infine la lunghezza del codice e il suo ultimo carattere, ricavato con un indice negativo.

Input di esempio: `lab2-pc17-2026`.

Output atteso:

```
laboratorio: LAB2
calcolatore: PC17
anno: 2026
lunghezza: 14, ultimo carattere: 6
```

Tempo stimato: 25 minuti

Esercizio 5 — Conteggio dei caratteri di una frase

Scrivere nella cella dell'esercizio 5 il programma che legge in ingresso una frase e conta, con un unico ciclo `for` sui caratteri, quante vocali, quante consonanti, quante cifre e quanti altri caratteri contiene. Il programma stampa i quattro conteggi e la percentuale di vocali sul totale dei caratteri, con un decimale. Le lettere accentate vanno considerate consonanti: è una semplificazione voluta, da dichiarare in un commento.

Input di esempio: `Il laboratorio 4BI dura 22 ore`.

Output atteso:

```
vocali: 12, consonanti: 10, cifre: 3, altri: 5
percentuale di vocali: 40.0%
```

Tempo stimato: 30 minuti

Esercizio 6 — Riparazione di un programma difettoso

Il programma seguente dovrebbe leggere la durata di una lezione in minuti e stampare il nome del docente, la durata espressa in ore intere e minuti, e i minuti che restano al netto di una pausa di dieci. Contiene quattro difetti: uno impedisce l'avvio del programma, uno lo interrompe alla prima operazione aritmetica, uno lo interrompe più avanti e uno produce un risultato sbagliato senza alcun messaggio.

```
durata = input("Durata della lezione in minuti: ")
docente = "Trentini"
pausa = 10

ore = durata / 60
minuti = durata % 60

if ore ⩾ 2
    print("lezione lunga")

print("docente:", Docente)
print("durata:", ore, "ore e", minuti, "minuti")
print("minuti al netto della pausa:", durata - pausa)
```

Copiare il programma nella cella dell'esercizio 6 e ripararlo, procedendo un difetto alla volta e rieseguendo dopo ogni correzione. Sopra il codice corretto vanno scritte quattro righe di commento, una per difetto, ciascuna nella forma «riga *n*: *messaggio dell'interprete oppure sintomo osservato*, causa, correzione». Il difetto silenzioso va descritto indicando che cosa si vede nell'uscita, dato che non produce alcun messaggio.

Input di esempio: `150`.

Output atteso:

```
lezione lunga
docente: Trentini
durata: 2 ore e 30 minuti
minuti al netto della pausa: 140
```

Tempo stimato: 25 minuti

Livello intermedio

Esercizio 7 — Analisi delle cifre di un numero

Scrivere nella cella dell'esercizio 7 il programma che legge in ingresso un intero non negativo e ne calcola, con un ciclo `while` che usa soltanto il resto e la divisione intera per dieci, la somma delle cifre, il numero di cifre e la cifra maggiore. Il numero zero va gestito correttamen-

te: ha una cifra, la somma è zero e la cifra maggiore è zero. Non è ammesso trasformare il numero in stringa.

Input di esempio: `48723`.

Output atteso:

```
cifre: 5
somma delle cifre: 24
cifra maggiore: 8
```

Tempo stimato: 30 minuti

Esercizio 8 — Conversione fra decimale e binario nei due sensi

Scrivere nella cella dell'esercizio 8 il programma che legge in ingresso un intero non negativo, ne costruisce la rappresentazione binaria come stringa con il metodo dei resti successivi, e la confronta con il risultato della funzione predefinita `bin`, stampando `verifica superata` oppure `verifica fallita`. Il programma legge poi una seconda stringa composta di soli zeri e uni e ne calcola il valore decimale scorrendone i caratteri da sinistra a destra, senza usare la conversione predefinita. Se la seconda stringa contiene caratteri diversi da `0` e `1`, il programma lo segnala e non calcola nulla.

Input di esempio: `2026`, poi `11010`.

Output atteso:

```
2026 in binario: 11111101010
verifica superata
11010 in decimale: 26
```

Tempo stimato: 40 minuti

Esercizio 9 — Ricerca sequenziale in un elenco di codici

L'elenco delle aule assegnate alle classi è memorizzato in una sola stringa, con le voci separate dal punto e virgola: è la forma con cui in Python si lavora finché non si conoscono le liste, e corrisponde all'array di stringhe usato in Java.

Scrivere nella cella dell'esercizio 9 il programma che, a partire dalla stringa `"A101;B204;C307;D410;E512"`, legge in ingresso un codice e ne cerca la presenza con una scansione voce per voce: il programma estrae le voci una alla volta con `find` e slicing e le confronta con il codice cercato, interrompendo la scansione appena lo trova. Va stampata la posizione della voce contando da uno, oppure il messaggio di codice assente. La ricerca è insensibile agli spazi agli estremi e alle maiuscole. Il caso «non trovato» va gestito con la clausola `else` del ciclo, senza variabili booleane di appoggio.

Input di esempio: `c307` e poi, in una seconda esecuzione, `Z999`.

Output atteso, nelle due esecuzioni:

```
C307 è la voce numero 3 dell'elenco
Z999 non è presente nell'elenco
```

Tempo stimato: 40 minuti

Esercizio 10 — Statistiche di una sequenza letta con sentinella

Scrivere nella cella dell'esercizio 10 il programma che legge in ingresso una sequenza di voti in virgola mobile, uno per riga, e termina quando l'utente digita `0`. Il programma calcola e stampa il numero di voti inseriti, il massimo, il minimo, la media con due decimali e quanti voti sono risultati sufficienti, cioè maggiori o uguali a 6. I voti fuori dall'intervallo da 1 a 10 vanno segnalati e ignorati senza interrompere la lettura. Se non viene inserito alcun voto valido il programma lo dichiara e non calcola le statistiche.

Massimo e minimo vanno inizializzati con il primo valore valido letto, non con una costante scelta a priori: è la stessa scelta corretta già adottata in Java.

Input di esempio: `7`, `5.5`, `15`, `9`, `6`, `8`, `0`.

Output atteso:

```
valore 15.0 fuori intervallo: ignorato
voti validi: 5
massimo: 9.00
minimo: 5.50
media: 7.10
sufficienze: 4
```

Tempo stimato: 45 minuti

Esercizio 11 — Palindroma con i due indici

Scrivere nella cella dell'esercizio 11 il programma che legge in ingresso una frase, ne costruisce la versione normalizzata tenendo soltanto le lettere e portandole in minuscolo, e verifica se è palindroma con il confronto fra due indici che si avvicinano dagli estremi verso il centro, interrompendo il ciclo al primo carattere diverso. Lo slicing con passo negativo va usato solo alla fine, per stampare una riga di controllo che confronta il risultato ottenuto con quello dato dal confronto `ripulita == ripulita[::-1]`. Il programma stampa anche quanti confronti sono stati eseguiti prima di decidere.

Input di esempio: `I topi non avevano nipoti`.

Output atteso:

```
testo normalizzato: itopinonavevanonipoti
palindroma: True
confronti eseguiti: 10
controllo con lo slicing: True
```

Tempo stimato: 40 minuti

Esercizio 12 — Occorrenze di una sottostringa senza il metodo count

Scrivere nella cella dell'esercizio 12 il programma che legge in ingresso un testo e una sottostringa da cercare, e conta quante volte la sottostringa compare nel testo usando `find` ripetutamente dentro un ciclo, senza chiamare `count`. Il programma stampa il numero di occorrenze e la posizione di ciascuna, contando da zero. Alla fine stampa la stessa cosa calcolata con `count`, come verifica del proprio risultato, e stampa il testo con tutte le occorrenze sostituite da un asterisco per ciascun carattere della sottostringa, ottenuto ricostruendo la stringa con lo slicing e la concatenazione, senza `replace`.

Attenzione al caso in cui la sottostringa cercata sia vuota: il programma deve rifiutarla con un messaggio prima di iniziare la ricerca.

Input di esempio: `la lampada della lavagna` e `la`.

Output atteso:

```
occorrenze: 4
posizioni: 0 3 14 17
verifica con count: 4
testo mascherato: ** **mpada del** **vagna
```

Tempo stimato: 45 minuti

Esercizio 13 — Traduzione di un metodo Java in Python

Il metodo seguente è stato scritto in Java al terzo anno. Riceve una stringa di voti espressi con una sola cifra, ignora i caratteri che non sono cifre, e restituisce la percentuale di voti sufficienti sul totale dei voti validi.

```java
public static double percentualeSufficienze(String voti) {
    int validi = 0;
    int sufficienti = 0;
    for (int i = 0; i < voti.length(); i++) {
        char c = voti.charAt(i);
        if (Character.isDigit(c)) {
            validi++;
            if (c - '0' >= 6) {
                sufficienti++;
            }
        }
    }
    if (validi == 0) {
        return 0.0;
    }
    return (double) sufficienti / validi * 100;
}
```

Scrivere nella cella dell'esercizio 13 la traduzione in Python, che legge la stringa dei voti in ingresso invece di riceverla come parametro e stampa i risultati invece di restituirli, perché la definizione di funzioni appartiene al modulo M4. Il programma stampa il numero di voti validi, il numero di sufficienze e la percentuale con un decimale; il caso in cui non ci sia alcun voto valido va dichiarato con un messaggio al posto della percentuale.

Sotto il codice vanno scritte quattro righe di commento che indicano, una per riga, come è stato tradotto il ciclo `for` con l'indice, che cosa sostituisce `voti.charAt(i)`, che cosa sostituisce l'espressione `c - '0'` e perché in Python non serve il cast a `double` prima della divisione.

Input di esempio: `7a8-6 9 0 5 4`.

Output atteso:

```
voti validi: 7
sufficienze: 4
percentuale di sufficienze: 57.1%
```

Tempo stimato: 45 minuti

Livello avanzato

Esercizio 14 — Ordinamento per conteggio delle cifre

In Java l'ordinamento di un array di numeri si è scritto con lo scambio fra elementi, tipicamente con l'algoritmo a bolle o per selezione. Senza le liste quella tecnica non è disponibile, ma per un insieme di valori che appartengono a un intervallo piccolo e noto esiste un'alternativa: con-

tare quante volte compare ciascun valore possibile e ricostruire il risultato in ordine. È l'idea dell'ordinamento per conteggio.

Scrivere nella cella dell'esercizio 14 il programma che legge in ingresso una stringa di sole cifre e ne stampa le cifre in ordine crescente, ottenendo il risultato con un ciclo `for` sulle dieci cifre possibili e con il conteggio delle occorrenze di ciascuna. Il programma stampa anche la distribuzione delle cifre presenti, una riga per cifra, con un istogramma fatto di asterischi. Se l'ingresso contiene caratteri che non sono cifre, il programma lo segnala e si ferma.

Nel commento finale della cella vanno scritte tre righe che confrontano questo procedimento con l'ordinamento a bolle scritto in Java: quale dei due esegue meno confronti, perché questo qui non è applicabile a un insieme di valori qualsiasi, e che cosa cambierà quando in M3 saranno disponibili le liste.

Input di esempio: `5382915`.

Output atteso:

```
cifre ordinate: 1235589
1 *
2 *
3 *
5 **
8 *
9 *
```

Tempo stimato: 50 minuti

Esercizio 15 — Verifica di un codice a barre EAN-13

Il codice a barre EAN-13 è composto da tredici cifre, di cui l'ultima è una cifra di controllo. La si calcola sommando le prime dodici cifre dopo averle moltiplicate alternativamente per uno e per tre, a partire dal peso uno per la prima cifra, e prendendo poi la differenza fra dieci e il resto della divisione per dieci di quella somma; se la differenza vale dieci, la cifra di controllo è zero.

Scrivere nella cella dell'esercizio 15 il programma che legge in ingresso un codice, ne verifica la lunghezza e la presenza di sole cifre, calcola la cifra di controllo a partire dalle prime dodici e la confronta con la tredicesima, dichiarando il codice valido o non valido. In caso di codice non valido va stampata anche la cifra di controllo corretta. Il programma stampa in ogni caso la somma pesata calcolata, perché serve a correggere l'esercizio.

Input di esempio: `8004980001332` e, in una seconda esecuzione, `8004980001335`.

Output atteso, nelle due esecuzioni:

```
somma pesata: 68
cifra di controllo attesa: 2
codice valido
```

```
somma pesata: 68
cifra di controllo attesa: 2
codice non valido: la cifra di controllo dovrebbe essere 2
```

Tempo stimato: 50 minuti

Esercizio 16 — Cifrario di Cesare

Scrivere nella cella dell'esercizio 16 il programma che legge in ingresso un testo e uno spostamento intero, e stampa il testo cifrato con il metodo di Cesare: ogni lettera viene sostituita dalla lettera che si trova nello spostamento indicato più avanti nell'alfabeto, ricominciando dall'inizio quando si supera la zeta. Le lettere maiuscole vanno portate in minuscolo prima della cifratura; gli spazi e i segni di punteggiatura restano invariati.

La posizione di una lettera nell'alfabeto si ottiene cercandola con `find` dentro la stringa `"abcdefghijklmnopqrstuvwxyz"`, e la lettera cifrata si ottiene indicizzando la stessa stringa con la posizione spostata, ridotta al resto della divisione per ventisei. Il programma stampa poi il testo decifrato applicando lo spostamento opposto, come controllo che la cifratura sia reversibile.

Lo spostamento va accettato anche negativo e anche maggiore di ventisei: il resto della divisione lo riporta comunque nell'intervallo giusto.

Input di esempio: `Laboratorio di informatica` e `5`.

Output atteso:

```
testo cifrato:   qfgtwfytwnt in nsktwrfynhf
testo decifrato: laboratorio di informatica
```

Tempo stimato: 55 minuti

Esercizio 17 — Ricerca binaria su un elenco ordinato di codici

In Java la ricerca binaria si è scritta su un array ordinato, con i due estremi che si avvicinano e l'elemento centrale confrontato con il valore cercato. L'algoritmo si trasferisce in Python senza alcuna modifica logica: cambia soltanto il modo di raggiungere la voce di posizione centrale, perché l'array non è disponibile e l'elenco è memorizzato in una sola stringa.

```
elenco = "A101;B204;C307;D410;E512;F615;G718"
```

Le voci sono ordinate alfabeticamente e hanno tutte quattro caratteri, quindi la voce di indice `i` occupa le posizioni da `i * 5` a `i * 5 + 3` della stringa e il numero delle voci si ricava dalla lunghezza complessiva, senza scriverlo a mano.

Scrivere nella cella dell'esercizio 17 il programma che legge in ingresso un codice, normalizzandolo con `strip` e `upper`, e lo cerca con la ricerca binaria. A ogni passo il programma stampa una riga con il numero del passo e la voce esaminata, contando le voci da uno. Al termine dichiara la posizione trovata oppure l'assenza del codice, e stampa il numero di confronti eseguiti insieme al numero di confronti che sarebbero serviti con la ricerca sequenziale dell'esercizio 9. Il confronto fra due codici si scrive con gli operatori `<`, `>` e `==` applicati direttamente alle stringhe, che seguono l'ordine alfabetico.

Input di esempio: `e512` e, in una seconda esecuzione, `Z999`.

Output atteso, nelle due esecuzioni:

```
passo 1: esaminata la voce 4 (D410)
passo 2: esaminata la voce 6 (F615)
passo 3: esaminata la voce 5 (E512)
E512 trovato in posizione 5
confronti eseguiti: 3
la ricerca sequenziale ne avrebbe richiesti 5
```

```
passo 1: esaminata la voce 4 (D410)
passo 2: esaminata la voce 6 (F615)
passo 3: esaminata la voce 7 (G718)
Z999 non è presente nell'elenco
confronti eseguiti: 3
la ricerca sequenziale ne avrebbe richiesti 7
```

Tempo stimato: 55 minuti

Esercizio 18 — Riepilogo di un registro di presenze

Il registro delle presenze di una settimana è memorizzato in una sola stringa: le righe sono separate dalla barra verticale e i campi di ogni riga dal punto e virgola, nell'ordine data, classe, ore di lezione e ore di assenza complessive.

```
registro = "2026-11-16;4BI;4;3|2026-11-17;4BI;2;0|2026-11-18;4BI;4;6|2026-11-19;4BI;2;1|
2026-11-20;4BI;4;2"
```

Scrivere nella cella dell'esercizio 18 il programma che scorre la stringa con `find` e slicing e produce un riepilogo con il totale delle ore di lezione, il totale delle ore di assenza, la percentuale di assenza con un decimale, il giorno con più assenze e il numero di giorni in cui le assenze hanno superato le due ore. Ogni riga del registro va stampata in una tabella incolonnata con la data nel formato giorno/mese, le ore di lezione, le ore di assenza e un asterisco nei

giorni oltre le due ore di assenza. Le colonne hanno rispettivamente ampiezza otto a sinistra, otto a destra e nove a destra; l'asterisco è preceduto da due spazi.

Il programma va scritto in modo da funzionare anche se al registro si aggiungono altre righe, senza modificare il codice: non è ammesso alcun riferimento al numero di righe scritto a mano.

Output atteso:

```
data     lezione  assenze
16/11          4        3  *
17/11          2        0
18/11          4        6  *
19/11          2        1
20/11          4        2
-------------------------
ore di lezione: 16
ore di assenza: 12
percentuale di assenza: 75.0%
giorno con più assenze: 18/11
giorni oltre le due ore: 2
```

Tempo stimato: 60 minuti

Criteri di valutazione

Ogni esercizio consegnato viene valutato in decimi secondo la griglia seguente. Il peso indicato vale per gli esercizi di livello intermedio e avanzato; per il livello base i criteri di leggibilità e di gestione dei casi limite pesano la metà e il punteggio viene riproporzionato.

Criterio	Peso	Descrittore per la valutazione piena
Correttezza del risultato	3	Il programma produce con i dati di esempio esattamente l'output atteso, compresi spazi, allineamenti e numero di decimali
Rispetto dei vincoli del modulo	2	Nessun costrutto dei moduli successivi: niente liste, tuple, dizionari, insiemi, `split` o `def`
Correttezza delle conversioni di tipo	1	Il risultato di `input` viene convertito dove serve; la divisione intera è usata dove il dato è intero
Gestione dei casi limite	2	Ingresso vuoto, valore zero, sequenza senza dati validi e valori fuori intervallo sono gestiti come richiesto dalla consegna
Leggibilità del codice	1	Nomi di variabile in italiano e conformi a PEP 8, indentazione di quattro spazi, commenti dove il procedimento non è ovvio
Uso di Git	1	Il lavoro è in commit distinti, con messaggi che richiamano il numero dell'esercizio

Negli esercizi 6 e 13, che chiedono di riparare un programma e di tradurne uno da Java, i commenti richiesti dalla consegna fanno parte del risultato e non sono un contorno: un programma corretto senza i commenti richiesti vale al massimo la metà del criterio di correttezza.

Sono considerati errori gravi, e comportano da soli la valutazione insufficiente indipendentemente dal resto: un programma che si interrompe con un errore dell'interprete sui dati di esempio della consegna; l'uso della divisione reale dove il dato richiede un intero, riconoscibile dalla comparsa di un `.0` di troppo nell'uscita; il risultato di un metodo di stringa calcolato e non assegnato; un ciclo `while` privo dell'istruzione che rende falsa la condizione; una cella consegnata senza essere mai stata eseguita, riconoscibile dall'assenza di uscita sotto il codice o dalla presenza di errori di sintassi.

Gli esercizi dal 14 al 18 concorrono alla valutazione del modulo con peso doppio rispetto agli altri. Del livello base vanno consegnati tutti e sei gli esercizi; del livello intermedio almeno cinque dei sette, fra i quali obbligatoriamente il 13; del livello avanzato almeno due dei cinque, di cui obbligatoriamente il 18.